Passthrough

611 Fifth Avenue, New York, NY 10022

Screening Test/Interview Questions for Software Engineer (Remote) Position

Instructions: Please precede all your answers with the question you are answering.
Use acronyms only after you've explained them.
Use correct spelling and grammar.

Candidate's Name: Neel Gali

1. Can you explain the concept of object-oriented programming?

   A. Object-oriented programming (OOP) is a programming paradigm that uses "objects" to design applications and computer programs.

   Class- a class is a blueprint or prototype for creating objects. A class defines
   - a set of attributes (data)
   - methods (functions).

   Objects-Objects are instances of classes, which can contain both data (attributes) and methods (functions).

   The four main principles of OOP are **encapsulation, abstraction, inheritance, and polymorphism.**

   - **Encapsulation** means that an object's data is hidden from the outside and can only be accessed through methods.
   - **Abstraction** allows a programmer to hide complex implementation details and show only the necessary features of an object.
   - **Inheritance** lets a new class inherit properties and behaviors from an existing class, promoting code reuse.
   - **Polymorphism** allows objects to be treated as instances of their parent class rather than their actual class, enabling a single interface to control access to a general class of actions.

Real world use-case: a banking system where objects like accounts, customers, and transactions are modeled as classes with properties and methods, promoting code reusability, modularity, and scalability.

2. How do you handle memory management in your code?

   A. Memory management in code can be handled through various steps This includes:
      - Using languages with built-in garbage collection (e.g., Java) that automatically manage memory allocation and deallocation.
      - In languages that require manual management (e.g., C++), I ensure to allocate and deallocate memory appropriately using smart pointers.
      - By using appropriate data structures and algorithms with optimal time and space.
      - Regularly monitoring the application to monitor memory usage and detect any anomalies or memory leaks.

3. Tell me about a time when you had to optimize code for better performance.

   A. In my role as a Full Stack Developer at Oklahoma State University, I was part of a team tasked with enhancing the performance and user experience of the student portal. One significant optimization project involved refactoring APIs and optimizing database queries.

      API - An API (Application Programming Interface) is a set of rules and protocols for building and interacting with software applications. It defines the methods and data formats that applications can use to communicate with each other, effectively enabling different software systems to exchange data and functionality seamlessly.

      Problem:
      The student portal was experiencing slow response times, especially when accessing and processing large datasets. This affected the overall user experience and satisfaction.

      Action:
      To address this issue, I took the following steps:

      Profiling and Identifying Bottlenecks:

I started by profiling the existing codebase to identify performance bottlenecks by logging, I pinpointed slow API endpoints and inefficient database queries that were causing delays.

Optimizing Database Queries:

Many of the slow queries were performing **full table scans or had suboptimal indexing.** I rewrote these queries to use more efficient joins, added **appropriate indexes**, and optimized the schema where necessary. For instance, a query fetching student records was taking several seconds due to a lack of indexing on key columns. By adding indexes and optimizing the query logic, we reduced the execution time from several seconds to milliseconds.

Refactoring APIs:

I refactored the API endpoints to reduce redundant data processing and improve data handling efficiency. This involved rewriting some endpoints to use more efficient algorithms and data structures.
For example, I replaced a nested loop in one of the data processing functions with a more efficient hash map approach, which significantly reduced the time complexity.

Testing and Validation:

Jest (JavaScript Testing Framework): A testing framework for JavaScript, commonly used for unit testing JavaScript based applications.
JUnit (Java Unit Testing Framework): A testing framework for Java, used for writing and running unit tests.

I conducted thorough testing, including unit testing with Jest for frontend components and JUnit for backend components, to ensure the changes did not introduce any new issues.

4. What is the difference between a stack and a queue?

A. A **stack** is a data structure that follows the **Last In, First Out (LIFO)** principle, meaning that the last element added to the stack is the first one to be removed. Common operations include **push** (to add an element to the top of the stack) and **pop** (to remove the top element).
Use Cases:
Function call management in programming (call stack).
Undo mechanisms in text editors.

Depth-first search (DFS) in graph algorithms.

A **queue**, on the other hand, follows the **First In, First Out (FIFO)** principle, meaning that the first element added is the first one to be removed. Common operations include **enqueue** (to add an element to the end of the queue) and **dequeue** (to remove the front element).
Use Cases:
Managing tasks in **print queues**.
Handling **requests on web servers**.
Breadth-first search (BFS) in graph algorithms.

5. How do you ensure code quality and maintainability?

   A. **Clean Code:** adhering to SOLID principles, Following Design Patterns to Design code in a modular and reusable manner. Encapsulate functionality into classes, functions, and modules with clear responsibilities and interfaces. Favor composition over inheritance.

   Documentation: Document code thoroughly using comments, inline documentation, and README files. Describe the purpose, functionality, and usage of classes, methods, and modules to aid understanding and maintenance.

   Code Reviews: Conduct regular code reviews to identify issues, improve readability, and enforce coding standards.

   Automated Testing: Implement unit tests, integration tests, and end-to-end tests using testing frameworks like JUnit, Jest, or Selenium. Ensure comprehensive test coverage to catch bugs early and validate functionality.

   Linting and Formatting: Use code linting tools (e.g., ESLint, Pylint) to enforce coding standards, catch syntax errors, and maintain consistent code style.

Passthrough
611 Fifth Avenue, New York, NY 10022

6. Have you worked with any design patterns? If so, which ones and how did they benefit your project?

   A. Yes, during my internship at ZeMoSo technologies, I developed project called **Virtual Incubation Center** - The Virtual Incubation Centre is a comprehensive platform designed to foster collaboration and innovation among users, mentors, and investors in the entrepreneurial ecosystem. Here's an in-depth description of its functionalities:

- **User Registration/Login:**
  Users, mentors, and investors can register and log in with distinct roles, each having specific privileges and access levels within the platform.

- **Project Registration:**
  Users can register projects seeking investment or funding by providing detailed project information such as project description, goals, target market, financial requirements, and timeline.

- **Mentorship Requests:**
  - Users can request mentorship for their projects, specifying the areas where they need guidance and support from experienced mentors.
  - Mentors can accept or reject mentorship requests based on their expertise and availability.

- **Investment/Funding Opportunities:**
  - Investors can view listed projects seeking investment/funding and express interest by sending emails or contacting project owners directly.
  - Projects seeking funding can provide detailed investment terms, funding requirements, expected returns, and other relevant financial information.

Design Patterns used:

- **Factory Method Pattern**:

  Usage: Use the Factory Method pattern to create instances of different user types (User, Investor, Mentor) based on the type of registration/login.

  Benefits: This pattern promotes code reusability and scalability by allowing the addition of new user types without modifying existing registration/login logic. It ensures a flexible and extensible user registration system.

- **Observer Pattern:**

  Usage: Apply the Observer pattern to notify mentors when a user submits a project request for mentorship or funding.

Benefits: By decoupling the notification mechanism, this pattern enables mentors to receive real-time updates about project requests. It enhances communication and collaboration between users and mentors.

- **Singleton Pattern:**

  Usage: Implement the Singleton pattern for the admin module to ensure there's only one instance managing user/mentor/investor details.

  Benefits: The Singleton pattern guarantees a single point of access to admin functionalities, such as adding, deleting, or editing user-related data. It helps maintain consistency and avoids duplicate admin instances.

- **Builder Pattern:**

  Usage: Use the Builder pattern to construct complex project objects with various details (e.g., project type, investment details, mentorship status) during project registration.

  Benefits: The Builder pattern simplifies the construction of project objects step by step, allowing flexibility in setting project attributes. It improves code readability and facilitates the creation of different project configurations.

7. Can you explain the difference between synchronous and asynchronous programming?

   A. **Synchronous** programming means that tasks are performed one after another, and each task must be completed before the next one begins. This can lead to inefficiencies, especially when tasks involve waiting (e.g., I/O operations).
   Asynchronous programming, on the other hand, allows tasks to run concurrently. When a task is performing a waiting operation, the program can continue to execute other tasks. This is **achieved using constructs like callbacks, promises, and async/await in various programming languages.** Asynchronous programming is particularly beneficial in applications that require high performance and responsiveness, such as **web servers and user interface applications**.

8. How do you approach debugging and troubleshooting complex issues?

A. **Understanding the problem:** Clearly defining the issue and gathering as much information as possible.
Reproducing the issue: Creating a minimal reproducible example to consistently trigger the issue.
Using debugging tools: Utilizing tools like debuggers, log analyzers, and performance profilers to gather insights into the problem.
Isolating the cause: Narrowing down the potential causes by systematically disabling parts of the code or using breakpoints.
Analyzing logs: Reviewing application logs to identify error messages and unusual behavior.
Consulting documentation and resources: Looking up relevant documentation, forums, and knowledge bases for possible solutions.
Collaboration: Above all, it is essential to seek help from colleagues or the development team when needed, to identify gaps in knowledge, missing pieces of information, when working on code written by someone else.

Here are a few bugs I handled in my work experience:
Persistence of Old Values in Edit Window:

- Recognizing the problem stemming from data not clearing properly in the edit details window, I delved into the frontend codebase to understand the data handling processes during clearing actions.
- Using debugging tools, I traced the flow of data from frontend to backend, identifying that the backend was not receiving update requests upon clearing values.
- Collaborating with backend developers, I implemented a solution to trigger backend updates when the clear action occurs, ensuring data consistency.

Property Filter Issue:

- The filter querying logic issue in a service necessitated thorough debugging of API calls, filter implementations, and data flow between frontend and backend.
- Through extensive payload monitoring and network call analysis, I identified discrepancies in filter queries and data processing.
- Implementing targeted changes in filter logic and API calls, coupled with rigorous UI testing, ensured correct property filtering and management within the service.

9. What is the importance of unit testing in software development?

   A. Let’s understand the importance of unit testing, considering a real-world example:
   Imagine a scenario where a team is developing an e-commerce platform that allows users to browse products, add them to cart, and checkout. One crucial component of this platform is the pricing module, responsible for calculating the total cost based on product prices, discounts, and shipping charges.

   With Unit Testing:

   Early Detection of Bugs:

   Unit tests are written for the pricing module to validate its calculations for different scenarios, such as applying discounts, adding shipping charges, and handling tax calculations.

   During development, the unit tests catch a bug where the shipping charges are not added correctly, leading to incorrect total cost calculations.
   The bug is fixed promptly because it's caught early, preventing it from propagating to other parts of the platform.

   Code Quality and Refactoring Safety:

   The unit tests act as a safety net when the pricing module undergoes refactoring to improve performance.
   After refactoring, running the unit tests ensures that the core functionality of the pricing module remains intact, despite changes in the codebase.

   Regression Prevention:

   As new features are added to the platform, the unit tests are run regularly to ensure that the pricing module continues to work correctly.
   If any changes inadvertently break the pricing calculations, the unit tests immediately flag the regression, allowing developers to fix it before it impacts users.

   Without Unit Testing:

   - Without unit tests, the pricing module's bugs might go unnoticed until integration testing or even after deployment.

Users might encounter issues during checkout, such as incorrect total costs, leading to a poor user experience and potential loss of sales.

- Refactoring the pricing module without unit tests poses a higher risk of introducing new bugs or breaking existing functionality.
- Without automated tests, regression issues in the pricing module may occur frequently as new code is added or modified.
- Each code change becomes a potential risk, as there's no automated way to verify the correct functioning of the pricing calculations.

10. Have you used any version control systems? If so, which ones and how do you handle branching and merging?

    A. Yes, I have used version control systems such as **Git**. In Git, I handle branching and merging as follows:

    - **Branching:** Creating separate branches for new features, bug fixes, and experiments to keep the main codebase stable.
    - **Feature branches:** Each feature or fix is developed in its own branch, named descriptively (e.g., feature/login, bugfix/issue-123).
    - **Merging:** Regularly merging changes from the main branch into the feature branches to keep them up to date. Once the feature is complete, merge the feature branch back into the main branch after a pull request.
    - **Pull requests:** Using pull requests to propose changes, which are then reviewed and discussed before merging.
    - **Conflict resolution:** Carefully resolving merge-conflicts by understanding the changes in each branch and testing thoroughly after the merge.

11. Can you explain the concept of multithreading and its advantages?

    A. Multithreading is a programming concept where multiple threads are created within a single process to perform tasks concurrently. Each thread can run independently and share the process's resources, such as memory.

    Advantages of multithreading include:

    Increased responsiveness: In interactive applications, multithreading can keep the application responsive by performing background tasks concurrently.
    Resource sharing: Threads within the same process can share resources more efficiently than separate processes.

Performance improvement: On multi-core processors, multithreading can improve performance by parallelizing tasks.
Simplified design: Certain applications, like those involving concurrent I/O operations or real-time processing, can be more easily designed using multithreading.
Example: - The design of the web server remains manageable and straightforward, thanks to the concurrency provided by multithreading.

12. How do you stay updated with the latest technologies and trends in software development?

    A. I do several things to stay updated in software development including:
    Reading: Following industry blogs, news sites, and publications like Medium, Hacker News, TLDR newsletters etc.
    Online courses and tutorials: Enrolling in courses on platforms like Coursera, Udemy, and edX, YouTube tutorials to learn new technologies and tools.
    Attending conferences and meetups: Participating in industry conferences, webinars like Google I/O, react developers conference, AWS reinvent etc.
    Networking: Engaging with the developer community on platforms like GitHub, Twitter, and LinkedIn.
    Experimenting: Building side projects and experimenting with new languages.

13. Have you worked with any cloud platforms or services? If so, which ones and what were the challenges you faced?

    A. **AWS Services Used:**

    S3 (Simple Storage Service): Used for storing and transferring data between Python scripts. Ensured data integrity and accessibility.

    Secrets Manager: Utilized for securely storing and managing credentials required for API integration. Enhanced security and reduced the risk of credential exposure.

    Lambda: Implemented Python scripts as serverless functions to fetch data from APIs, process it, and perform necessary actions. Enabled scalable and cost-effective execution.

CloudWatch: Monitored and logged events, errors, and performance metrics of Lambda functions. Facilitated debugging and performance optimization.

Event Bridge: Scheduled and triggered Lambda functions based on predefined events or schedules. Orchestrated the workflow and ensured timely execution.

Challenges Faced:

Integration Complexity: Integrating data across multiple systems (UKG, Jump Cloud, Azure AD) while ensuring data consistency and synchronization was challenging. Required careful planning and robust error handling.

Security Concerns: Handling sensitive credentials and ensuring secure data transmission and storage (using Secrets Manager, HTTPS, encryption) were critical to prevent data breaches.

API Rate Limiting: Dealing with API rate limits and optimizing API calls to avoid throttling and ensure efficient data retrieval and processing.

Event Orchestration: Designing and implementing the workflow to trigger events based on changes in the UKG database and schedule regular data synchronization tasks required thorough understanding of event-driven architecture and scheduling mechanisms.

Azure AD Integration:

Azure Active Directory: Used for user authentication, access control, and identity management within the Azure ecosystem.

API Integration: Utilized Azure AD Graph API or Microsoft Graph API to interact with Azure AD programmatically, managing users, groups, licenses, and other directory objects.

Azure AD Connect: Possibly used for syncing on-premises directories with Azure AD, ensuring a unified identity across cloud and on-premises environments.

Challenges and Considerations:

Data Consistency: Ensured that user data (e.g., user attributes, group memberships, licenses) remained consistent across UKG, Jump Cloud, and Azure AD. Implemented synchronization logic to handle updates, additions, and deletions in real-time or at scheduled intervals.

API Permissions: Configured appropriate API permissions and roles within Azure AD to grant necessary access for the integration workflow. Managed OAuth tokens or service principals securely for API authentication.

Attribute Mapping: Mapped and transformed user attributes and data between different systems to ensure accurate representation and functionality (e.g., mapping UKG employee data to Azure AD user profiles).

License Provisioning: Managed Azure AD licenses based on user roles or group memberships, ensuring compliance and cost-effectiveness.

Security and Compliance: Adhered to Azure AD security best practices, including multi-factor authentication (MFA), conditional access policies, and role-based access control (RBAC). Ensured compliance with regulatory standards (e.g., GDPR, HIPAA) regarding user data handling and privacy.

Error Handling and Logging: Implemented robust error handling mechanisms to handle API errors, network failures, and data inconsistencies. Logged events, errors, and audit trails for troubleshooting and compliance purposes.

14. Can you describe the steps you take to ensure secure coding practices?

A. Ensuring secure coding practices involves several steps:

Input Validation: Validate all user inputs to prevent injection attacks such as SQL injection, XSS (Cross-Site Scripting), and command injection.

Authentication and Authorization: Implement strong authentication mechanisms (e.g., OAuth, JWT) and role-based access control (RBAC) to ensure only authorized users access sensitive data and functionalities.

Secure Communication: Use HTTPS for secure communication over the network and encrypt sensitive data at rest using strong encryption algorithms.

Avoid Hardcoding Secrets: Store sensitive information such as API keys, passwords, and tokens securely, preferably in environment variables or secure vaults.

Update Dependencies: Regularly update dependencies, libraries, and frameworks to patch vulnerabilities and maintain a secure codebase.

Secure Configuration: Configure servers, databases, and services securely by disabling unnecessary features, using secure protocols, and applying security patches promptly.

Error Handling: Implement proper error handling to **avoid leaking sensitive information** in error messages and **logs**.

Security Testing: Conduct regular security assessments, penetration testing, and code reviews to identify and mitigate security vulnerabilities early in the development lifecycle.

Compliance: Adhere to industry standards and regulations (e.g., GDPR, HIPAA, PCI DSS) relevant to the application's domain to ensure data protection and regulatory compliance.

15. How do you handle and prioritize multiple tasks or projects simultaneously?

A. I usually follow the following principles:

Task Identification: Clearly identify all tasks and projects, breaking them down into smaller, manageable units, milestones or sprints to achieve incremental progress and maintain motivation.

Use Task Management Tools: Utilize task management tools like **Jira, Trello, or Asana** to **organize tasks, track progress, and collaborate** with team members.

Prioritization: Prioritize tasks based on urgency, impact, deadlines, dependencies, and strategic importance.

Time Management: Allocate specific time blocks for each task or project, considering their priority and complexity.

Focus on Deadlines: Meet deadlines by setting realistic timelines, avoiding procrastination, and monitoring progress regularly.

Communication: Communicate effectively with stakeholders, team members, and managers to align priorities, share progress, and address any challenges or bottlenecks.

Delegate and Collaborate: Delegate tasks, when possible, collaborate with team members, and leverage their expertise to achieve collective goals efficiently.

Responsibilities:

- Develop, test, and enhance backend systems and services using Java to improve visibility and tracking of product delivery.
- Collaborate with cross-functional teams to design and implement scalable and reliable solutions that enhance customer experience.
- Ability to follow test-driven development (TDD) practices.
- Design and implement data models and databases for efficient data storage and retrieval, contributing to enhanced product tracking capabilities.
- Lead and participate in code reviews.
- Ensure backend systems' security, scalability, and performance while prioritizing product visibility and tracking.
- Write clean, maintainable, and well-documented code that aligns with the team's focus on improved product visibility and tracking.

Can you handle all of these duties effectively?

A. Yes, I can handle these responsibilities effectively, I can:

Develop, Test, and Enhance Backend Systems: I have experience developing backend systems and services using Java. I follow best practices and design patterns to ensure scalability, reliability, and maintainability. I also prioritize testing throughout the development process, including unit tests, integration tests, and end-to-end tests.

Collaborate with Cross-Functional Teams: I excel in collaborating with cross-functional teams to design and implement scalable solutions. I understand the importance of communication and teamwork in delivering successful projects that enhance customer experience.

Test-Driven Development (TDD) Practices: I have expertise in following test-driven development practices, where I write tests for the actual code. This

approach helps in writing clean, modular, and testable code, reducing bugs and improving overall code quality.

Design and Implement Data Models and Databases: I am proficient in designing data models and databases for efficient data storage and retrieval. I use technologies like SQL and NoSQL databases to contribute to enhanced product tracking capabilities.

Lead and Participate in Code Reviews: I have experience leading and participating in code reviews. I understand the importance of code reviews in identifying issues, ensuring code quality, and sharing knowledge among team members.

Ensure Backend Systems' Security, Scalability, and Performance: I prioritize backend systems' security, scalability, and performance by implementing best practices, security measures, and performance optimizations. This includes securing APIs, managing resources efficiently, and optimizing database queries.

Write Clean, Maintainable, and Well-Documented Code: I emphasize writing clean, maintainable, and well-documented code that aligns with the team's focus on improved product visibility and tracking. This includes following coding standards, documenting codebase, and writing clear comments and documentation.

Overall, I am confident in my ability to handle these responsibilities effectively and contribute positively to the team's goals of enhancing product visibility, tracking, and customer experience.

Passthrough
611 Fifth Avenue, New York, NY 10022